TP 1 — Découvrir la boîte à outils MLOps

Durée : 2 h · Partie 2 du cours · binôme de 2

Ce premier TP installe la chaîne d'outils qui servira dans les onze TP suivants, puis fait toucher du doigt les trois fonctions d'une boîte à outils MLOps : le suivi des exécutions, la gestion des versions de modèles et l'automatisation des enchainements. Version Windows : toutes les commandes sont écrites pour PowerShell 5.1 ou 7, livré avec Windows 10 et 11. Ouvrez « Windows PowerShell », jamais l'invite de commandes cmd.exe. Les blocs @' ... '@ sont des here-strings : le marqueur de fermeture '@ doit impérativement se trouver en début de ligne, sans espace devant, et le bloc entier se colle en une seule fois.

Objectifs pédagogiques

- Installer et vérifier une chaîne d'outils MLOps complète sur son poste
- Journaliser une exécution d'entraînement et la retrouver dans une interface web
- Comparer plusieurs exécutions et sélectionner la meilleure sur une métrique
- Enregistrer un modèle dans un registre et lui affecter un alias
- Automatiser l'enchaînement des commandes par un script unique

Prérequis

- Python 3.10 ou plus, vérifiable par python --version
- Git installé et configuré (git config user.name et user.email)
- Un terminal et un éditeur de code (VS Code conseillé)
- Environ 2 Go d'espace disque libre

Installation

Toutes les commandes de ce TP se lancent depuis le dossier tp-mlops, dans une fenêtre Windows PowerShell ouverte sur ce dossier.

terminal

```
mkdir tp-mlops; cd tp-mlops                # dossier de travail du module
python -m venv .venv                       # environnement Python isole
.venv\Scripts\Activate.ps1                 # activation (PowerShell)
pip install --upgrade pip -q
pip install mlflow dvc scikit-learn pandas numpy joblib
New-Item -ItemType Directory -Force data, models, src | Out-Null
mlflow --version                           # verifie l'installation
dvc --version
# si Activate.ps1 est refuse par la strategie d'execution :
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
```

RÉSULTAT ATTENDU

```
mlflow, version 3.x
3.6x.x

L'invite du terminal commence maintenant par (.venv).
```

Déroulement

TP 1 — Découvrir la boîte à outils MLOps

Durée : 2 h · Partie 2 du cours · binôme de 2

Étape 1 — Générer le jeu de données du module

Ce jeu de transactions synthétique sera réutilisé dans tous les TP. La graine fixe garantit que toute la promo travaille sur exactement les mêmes données.

terminal

```
@'
import numpy as np, pandas as pd
rng = np.random.default_rng(7)                # graine fixe : meme jeu pour toute la promo
n = 5000                                      # nombre de transactions
montant = np.round(rng.lognormal(6.2, 1.1, n), 2)     # beaucoup de petits montants
anciennete = rng.integers(0, 120, n)                  # anciennete du client en mois
freq = rng.poisson(3, n) + 1                          # transactions sur 24 h
nuit = rng.binomial(1, 0.18, n)                       # paiement entre 0 h et 5 h
score = 2.2*(np.log10(montant)-3.0) - anciennete/60 + (freq-3)/3 + nuit*1.2
fraude = ((score + rng.normal(0, 0.35, n)) > 0.6).astype(int)   # etiquette 0 / 1
pd.DataFrame({'montant': montant, 'anciennete': anciennete, 'freq_24h': freq,
              'nuit': nuit, 'fraude': fraude}).to_csv('data/transactions.csv', index=False)
print('data/transactions.csv ecrit :', n, 'lignes,', fraude.sum(), 'fraudes')
'@ | Set-Content -Encoding UTF8 donnees.py
python donnees.py
```

RÉSULTAT ATTENDU

```
data/transactions.csv ecrit : 5000 lignes, 568 fraudes
```

Étape 2 — Démarrer le serveur de suivi MLflow

Le serveur stocke les exécutions dans une base SQLite et les artefacts dans un dossier. Laissez ce terminal ouvert pendant tout le TP et ouvrez-en un second pour la suite.

terminal 1

```
mlflow server --host 127.0.0.1 --port 5000 `
  --backend-store-uri sqlite:///mlflow.db `
  --default-artifact-root ./mlartifacts
```

RÉSULTAT ATTENDU

```
[INFO] Listening at: http://127.0.0.1:5000

Ouvrez cette adresse dans un navigateur : l'interface s'affiche, sans experimentation.
```

Si ça coince : les versions recentes de MLflow refusent un stockage sur simple dossier (./mlruns). Le paramètre sqlite:///mlflow.db est obligatoire.

Si ça coince : si le navigateur affiche « Refused to exécute script ... MIME type ('text/plain') », le registre Windows associe un mauvais type aux fichiers .js. Corrigez-le sans droits administrateur : New-Item -Path 'HKCU:\Software\Classes\.js' -Force ; Set-ItemProperty -Path 'HKCU:\Software\Classes\.js' -Name 'Content Type' -Value 'text/javascript'. Fermez tous les terminaux, rouvrez-en un, relancez le serveur et rechargez la page avec Ctrl+F5. L'API de suivi, elle, fonctionne déjà : vous pouvez enchaîner les étapes suivantes.

Étape 3 — Premier entraînement instrumente

La fonction autolog capture automatiquement les hyperparamètres, les métriques et le modèle. C'est la maniere la plus rapide de rendre une expérimentation traçable.

src/train.py

TP 1 — Découvrir la boîte à outils MLOps

Durée : 2 h · Partie 2 du cours · binôme de 2

```
@'
import sys, mlflow, mlflow.sklearn, pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
n_estimators = int(sys.argv[1]) if len(sys.argv) > 1 else 300   # 1er argument
max_depth    = int(sys.argv[2]) if len(sys.argv) > 2 else 12    # 2e argument
mlflow.set_tracking_uri('http://127.0.0.1:5000')   # serveur de l'etape 2
mlflow.set_experiment('tp1-decouverte')            # regroupe les executions
mlflow.sklearn.autolog()                           # capture automatique
df = pd.read_csv('data/transactions.csv')
X, y = df.drop(columns='fraude'), df['fraude']     # entrees / cible
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=42)
with mlflow.start_run(run_name=f'rf-{n_estimators}-{max_depth}'):
    m = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth,
                               random_state=42).fit(Xtr, ytr)
    f1 = f1_score(yte, m.predict(Xte))
    mlflow.log_metric('f1_test', f1)               # metrique de comparaison
    print('F1 =', round(f1, 4))
'@ | Set-Content -Encoding UTF8 src/train.py
python src/train.py 200 8
```

RÉSULTAT ATTENDU

```
F1 = 0.8549

Dans l'interface, l'experimentation tp1-decouverte contient une execution
nommee rf-200-8, avec ses parametres, ses metriques et son modele.
```

Étape 4 — Lancer une série d'exécutions et les comparer

Cinq exécutions comparables, obtenues en faisant varier deux hyperparamètres.

terminal 2

```
foreach ($p in '100 4', '200 8', '300 12', '500 16', '600 24') {
    $a = $p -split ' '                      # decoupe '200 8' en deux arguments
    python src/train.py $a[0] $a[1]
}
```

RÉSULTAT ATTENDU

```
F1 = 0.6634
F1 = 0.8549
F1 = 0.8636
F1 = 0.8669
F1 = 0.8631

Dans l'interface : cochez les cinq executions puis cliquez sur Compare.
Le gain s'ecrase apres 300 arbres.
```

Étape 5 — Enregistrer la meilleure exécution dans le registre

Le registre remplace l'echange de fichiers .pkl par un catalogue de versions. Repérez dans l'interface l'exécution de meilleur F1 et copiez son Run ID.

terminal 2

TP 1 — Découvrir la boîte à outils MLOps

Durée : 2 h · Partie 2 du cours · binôme de 2

```
@'
import mlflow
from mlflow import MlflowClient
mlflow.set_tracking_uri('http://127.0.0.1:5000')
RUN_ID = 'collez-ici-votre-run-id'                      # visible dans l'interface
mv = mlflow.register_model(f'runs:/{RUN_ID}/model', 'fraude-tp1')
MlflowClient().set_registered_model_alias('fraude-tp1', 'production', mv.version)
print('version', mv.version, 'promue en production')
'@ | python -
```

RÉSULTAT ATTENDU

```
Successfully registered model 'fraude-tp1'.
version 1 promue en production

Onglet Models de l'interface : le modele apparait avec l'alias @production.
```

Étape 6 — Recharger le modèle par son alias

C'est le geste que fera le service d'inférence du TP 10 : il ne connait ni le chemin du fichier, ni le numéro de version.

terminal 2

```
@'
import mlflow, pandas as pd
mlflow.set_tracking_uri('http://127.0.0.1:5000')
m = mlflow.pyfunc.load_model('models:/fraude-tp1@production')   # chargement par ALIAS
X = pd.DataFrame([{'montant': 8500, 'anciennete': 3, 'freq_24h': 9, 'nuit': 1}])
print('prediction :', m.predict(X))
'@ | python -
```

RÉSULTAT ATTENDU

```
prediction : [1]        # 1 = transaction suspecte
```

Étape 7 — Premier contact avec le versioning de données

DVC sera étudié en detail au TP 4. Ces trois commandes suffisent ici a voir ce qu'il fait : il sort le fichier volumineux de Git et le remplace par un pointeur.

terminal 2

```
git init                                  # depot Git du projet
dvc init                                  # initialise DVC par-dessus Git
dvc add data/transactions.csv             # sort le CSV de Git, cree le pointeur
Get-Content data/transactions.csv.dvc     # le pointeur : empreinte, taille, chemin
(Get-Item data/transactions.csv.dvc).Length   # moins de 100 octets
```

RÉSULTAT ATTENDU

```
outs:
- md5: 6203c87193e8cda3ca48b453c49dbca0
  size: 80901
  hash: md5
  path: transactions.csv

97 octets pour un fichier de 80 Ko.
```

TP 1 — Découvrir la boîte à outils MLOps

Durée : 2 h · Partie 2 du cours · binôme de 2

Étape 8 — Automatiser l'enchaînement

Troisième fonction d'une boîte à outils MLOps : ne plus rien taper à la main. Ce script rejoue l'intégralité du TP en une commande.

run_tp1.sh

```
@'
# rejoue l'integralite du TP en une commande
python donnees.py                          # 1. donnees
foreach ($p in '100 4', '200 8', '300 12', '500 16', '600 24') {
    $a = $p -split ' '
    python src/train.py $a[0] $a[1]        # 2. serie d'entrainements
}
Write-Host 'TP1 termine - consultez http://127.0.0.1:5000'
'@ | Set-Content -Encoding UTF8 run_tp1.ps1
.\run_tp1.ps1                              # execution du script
```

RÉSULTAT ATTENDU

Les cinq executions se relancent d'affilee et apparaissent dans l'interface.
C'est la brique de base de l'automatisation etudiee au TP 2.

Questions du compte rendu

1. Quelle différence faites-vous entre un paramètre et une métrique dans MLflow ? Donnez un exemple de chaque.
2. Deux exécutions ont un F1 très proche mais des durees très différentes. Laquelle promouvez-vous, et pourquoi ?
3. A quoi sert l'alias production plutot que le numéro de version dans le code du service d'inférence ?
4. Que contient exactement le fichier .dvc, et pourquoi Git ne doit-il pas suivre le CSV lui-même ?
5. Citez deux informations qu'il faudrait journaliser en plus pour qu'une exécution soit vraiment reproductible.

Livrables

- Le dossier tp-mlops avec un depot Git initialise et un premier commit
- Une capture de la comparaison des cinq exécutions dans MLflow
- Une capture de l'onglet Models montrant la version enregistrée et son alias
- Le script run_tp1.sh fonctionnel
- Un compte rendu d'une page repondant aux cinq questions

Barème

Critère	Points
Environnement installe et serveur MLflow fonctionnel	4
Cinq exécutions journalisées et comparees	5
Modèle enregistre, alias pose, rechargement démontré	5
Script d'automatisation qui rejoue le TP	3

TP 1 — Découvrir la boîte à outils MLOps

Durée : 2 h · Partie 2 du cours · binôme de 2

Critère	Points
Compte rendu et réponses aux questions	3